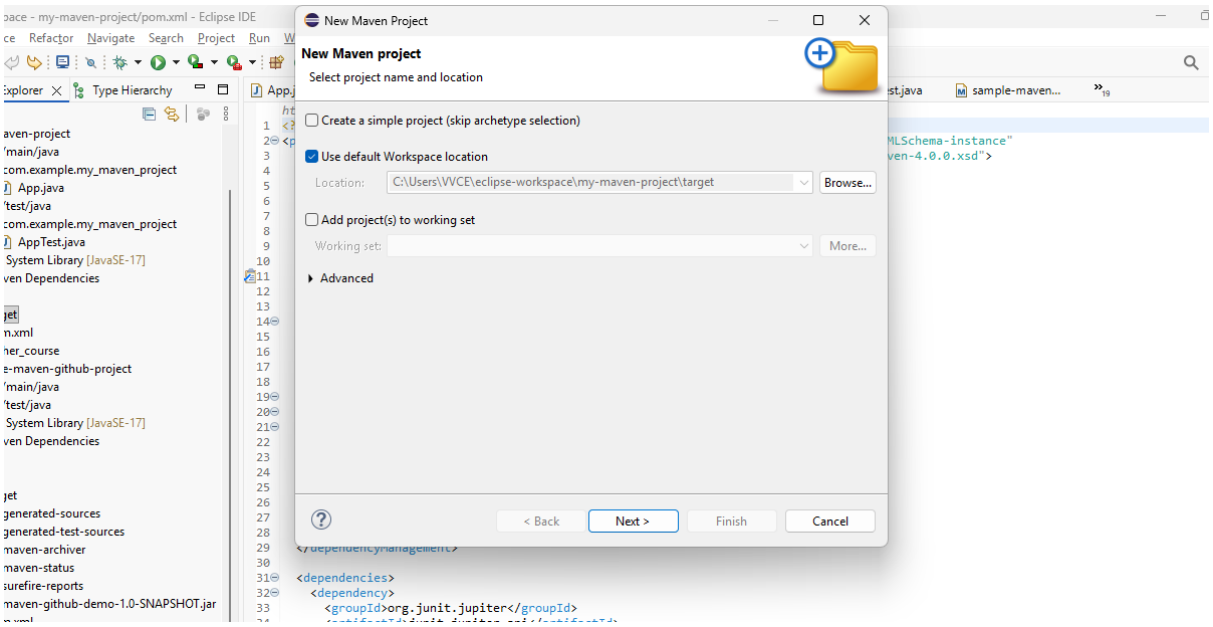
Experiment 3

Step 1: Create a Maven Project

In Eclipse:

File → New → Maven Project



Step 2: Select Maven Project

Click **Next**.

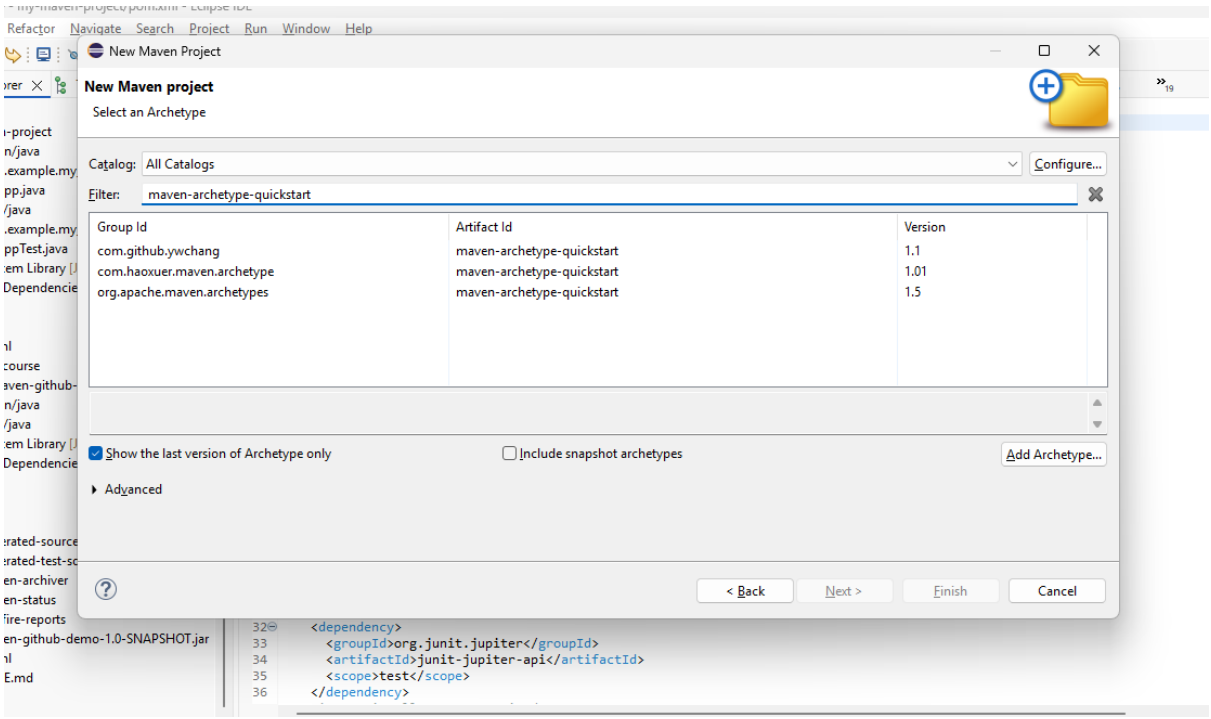
Step 3: Select Maven Archetype

In the **Filter** field, type:

maven-archetype-quickstart

Select the required quickstart archetype, **version 1.5**, and click **Next**.

Do not skip archetype selection if you want the project to be created using maven-archetype-quickstart.



Step 4: Enter Project Details

Enter:

Field	Value
Group Id	com.example
Artifact Id	maven-github-demo
Version	1.0-SNAPSHOT
Package	com.example

Click **Finish**.

Step 5: Type Y if Asked

Step 6: Maven Project Structure

Your project should look approximately like this:

maven-github-demo

```
|
├── src
|   ├── main
```

```
│   │   └── java
│   │       └── com.example
│   │           └── App.java
│   │
│   └── test
│       └── java
│           └── com.example
│               └── AppTest.java
│
├── pom.xml
└── ...
```

Maven Project should be created with App.java and AppTest.java file

You can add additional .java files under the appropriate src/main/java or src/test/java folders.

Step 7: Compile and Package the Project

Right-click the project:

Run As → Maven Build...

In **Goals**, enter clean package

Then click **Run**.

clean

↓

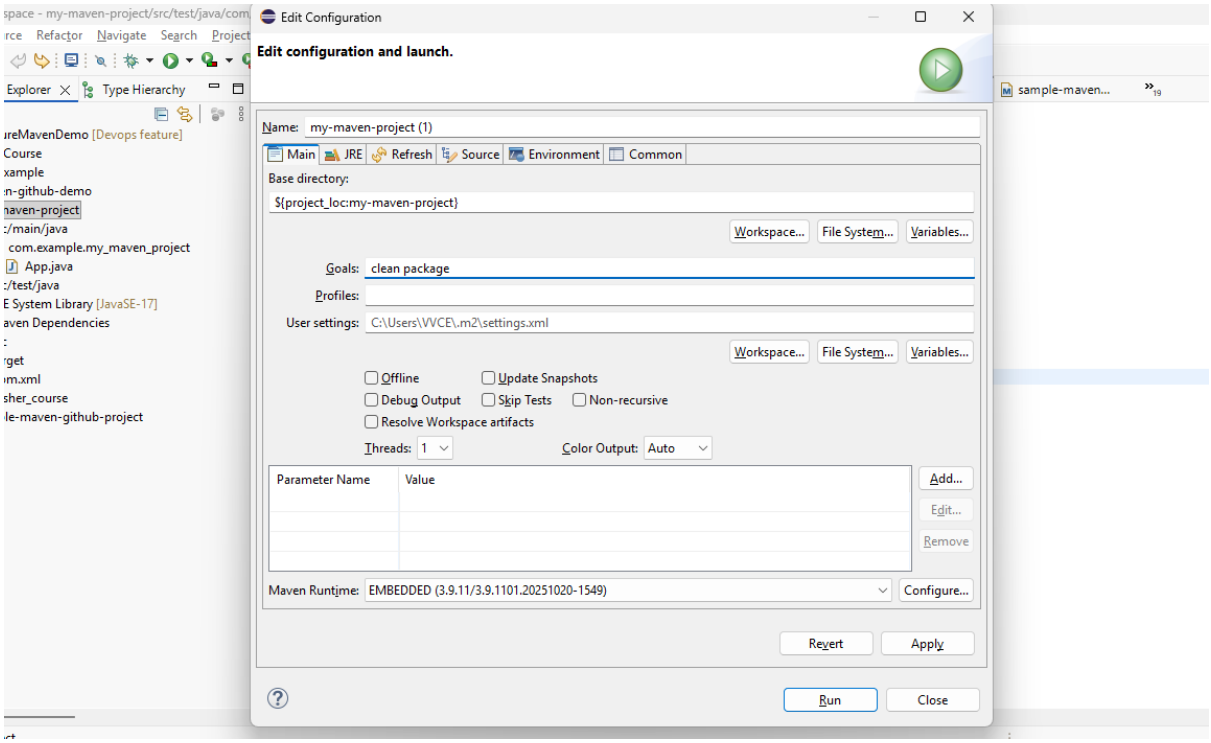
compile

↓

test

↓

package



Step 8: Run Maven Test

If you specifically want to execute the test cases:

Right-click the project:

Run As → Maven Test

Step 9: Refresh the Project

After Maven finishes:

Right-click Project → Refresh

Step 10: Check the target Folder

Step 11: Under Target folder .jar file should be Visible

maven-github-demo

```
│
├── src
├── pom.xml
└── target
```

```
    ├── classes
    ├── test-classes
    ├── surefire-reports
    └── maven-github-demo-1.0-SNAPSHOT.jar
```

The important file is:

maven-github-demo-1.0-SNAPSHOT.jar

Step 11: Create GitHub Actions Workflow

Under the **root folder** of your Maven project, create:

```
.github
└── workflows
    └── maven.yml
```

Step 12: Add maven.yml

Put this inside maven.yml:

```
name: Maven Build

on:
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]

jobs:
  build:

    runs-on: ubuntu-latest
```

```
steps:
  - name: Checkout source code
    uses: actions/checkout@v4

  - name: Set up JDK
    uses: actions/setup-java@v4
    with:
      java-version: '17'
      distribution: 'temurin'
      cache: maven

  - name: Build with Maven
    run: mvn clean package
```

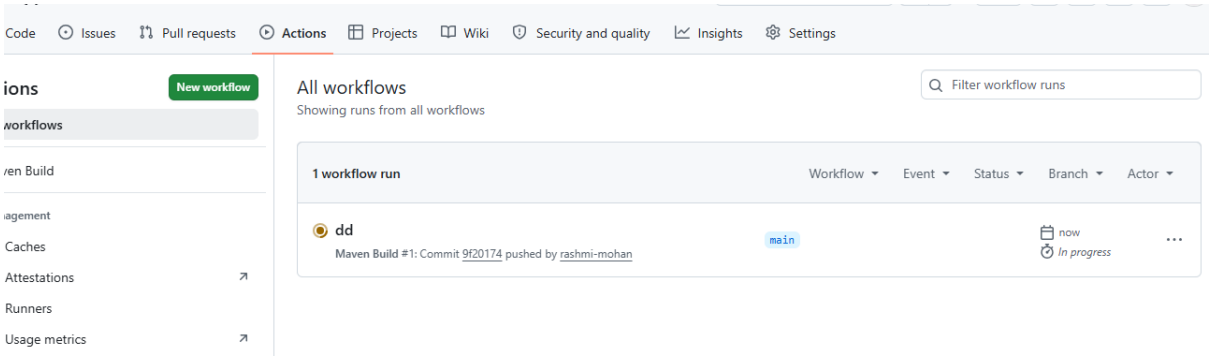
This tells GitHub:

1. Download your source code.
2. Install/configure Java.
3. Configure Maven dependency caching.
4. Run: mvn clean package

Step 13: Create GitHub Repository

Step 14: Initialize Git ->Add all project files: ->commit ->push the project to GitHub

Step 15: GitHub Actions Automatically Starts



Step 16: What Happens When You Make Changes?

Suppose you modify the file and push the changes to github

GitHub Actions automatically starts **another Maven build**.

Step 17: If the Build Fails

If GitHub Actions shows a red ❌, go to:

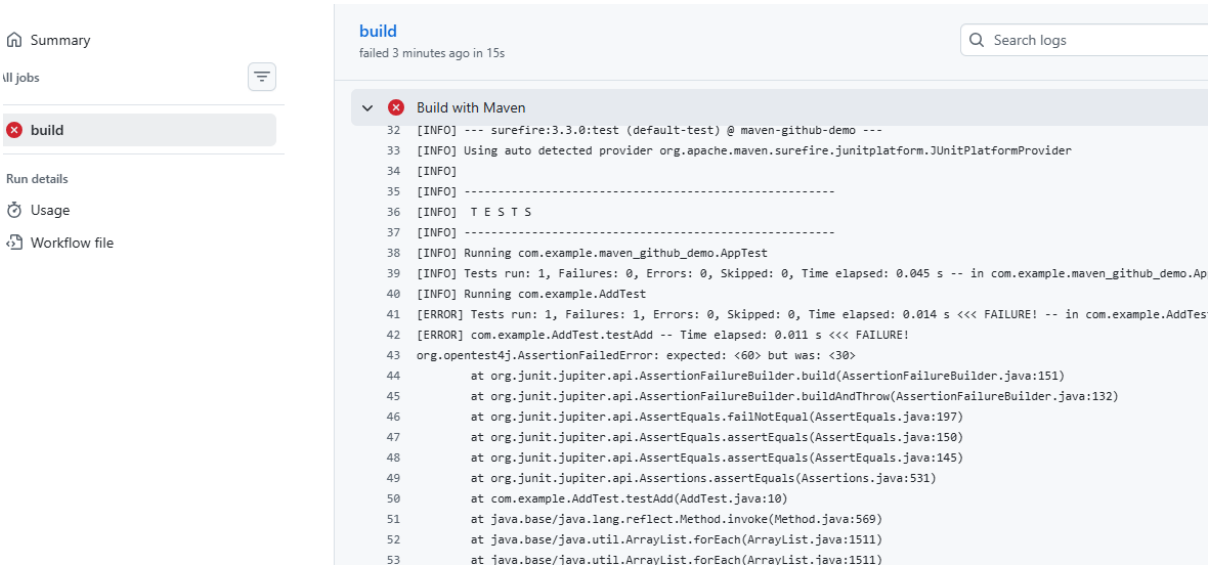
GitHub → Actions

Click the failed workflow run.

Maven Build └── ✕ build

Click **build**.

Look through the console output.


Summary
ll jobs
build
Run details
Usage
Workflow file
build
failed 3 minutes ago in 15s
Search logs
Build with Maven
[INFO] --- surefire:3.3.0:test (default-test) @ maven-github-demo ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] ---
[INFO] T E S T S
[INFO] ---
[INFO] Running com.example.maven_github_demo.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.045 s -- in com.example.maven_github_demo.Ap
[INFO] Running com.example.AddTest
[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 0.014 s <<< FAILURE! -- in com.example.AddTes
[ERROR] com.example.AddTest.testAdd -- Time elapsed: 0.011 s <<< FAILURE!
org.opentest4j.AssertionFailedError: expected: <60> but was: <30>
at org.junit.jupiter.api.AssertionFailureBuilder.build(AssertionFailureBuilder.java:151)
at org.junit.jupiter.api.AssertionFailureBuilder.buildAndThrow(AssertionFailureBuilder.java:132)
at org.junit.jupiter.api.AssertEquals.failNotEqual(AssertEquals.java:197)
at org.junit.jupiter.api.AssertEquals.assertEquals(AssertEquals.java:150)
at org.junit.jupiter.api.AssertEquals.assertEquals(AssertEquals.java:145)
at org.junit.jupiter.api.Assertions.assertEquals(Assertions.java:531)
at com.example.AddTest.testAdd(AddTest.java:10)
at java.base/java.lang.reflect.Method.invoke(Method.java:569)
at java.base/java.util.ArrayList.forEach(ArrayList.java:1511)
at java.base/java.util.ArrayList.forEach(ArrayList.java:1511)